

ENGINEERING DEEP-DIVE

DocuBrain

The Ultimate System-Design & Interview Revision Guide —
every endpoint, every tool, every concept, end to end.

FastAPI · async

PostgreSQL · asyncpg

SQLAlchemy 2.0

Celery · Redis

PgBouncer

Pinecone · RAG

Gemini · LangChain

Nginx

Docker

Terraform · AWS

GitHub Actions

Prometheus

k6

A complete walkthrough of the DocuBrain AI Study Workspace backend.

Written to be read cover-to-cover the night before a systems / backend interview.

Author: Rajit · **Scope:** architecture, request flows, tooling, and a 100+ question Q&A bank.

What's Inside

Read it top to bottom, or jump to the section an interviewer is poking at.

1. The 30-Second Pitch & The Big Picture

- What DocuBrain is, the end-to-end architecture, the request lifecycle

2. The Tech Stack — Every Tool, Why It Exists, How We Use It

- FastAPI · Uvicorn · PostgreSQL · asyncpg vs psycopg2 · SQLAlchemy 2.0 · Alembic
- PgBouncer · Redis (4 roles) · Celery · Pinecone · Gemini · LangChain · MinIO/S3
- Nginx · Docker · Prometheus · JWT stack · Terraform · GitHub Actions · k6

3. Endpoint-by-Endpoint Deep Flows

- Auth (register / login / refresh / logout) · Documents (upload / list / delete)
- The Celery worker pipeline · Chat (sessions + SSE streaming RAG)
- Quiz (generate / attempt / score) · Progress · Analytics · WebSocket notifications

4. Cross-Cutting Patterns (the stuff that impresses)

- Async connection discipline · Circuit breaker · Two-tier rate limiting
- RBAC policy layer · Append-only audit log · Correlation IDs · Read replica · RAG internals

5. The Roadmap — Phases 0→5, What Was Built & Why

6. The Concept & Interview Q&A Bank (100+ questions)

- Async/event loop/GIL · DB & pooling · Auth · RAG/vectors · Distributed systems
- Caching · Security/OWASP · Observability · Networking · Scaling & trade-offs

HOW TO USE THIS GUIDE

Each tool and endpoint section ends with the **exact questions an interviewer asks** and a tight model answer. If you can explain Sections 1, 4, and the Q&A bank out loud, you can defend this project against any backend interviewer.

The Big Picture

If you only memorise one page before the interview, make it this one.

The 30-second pitch

DocuBrain is an AI study workspace. A user uploads a PDF; the system extracts its text, splits it into chunks, turns each chunk into a vector embedding, and stores those vectors in a vector database. The user can then (a) **chat** with the document — questions are answered by a large language model that is fed only the most relevant chunks (this is **RAG**, Retrieval-Augmented Generation), and (b) auto-generate **quizzes and flashcards** from the document, take them, and track their **progress and analytics** over time.

The interesting part is not the AI — it's that the whole thing is built like a **production backend**: async API, background workers, connection pooling, circuit breakers, rate limiting, structured logging, metrics, real-time notifications, containerised, with infrastructure-as-code and a CI/CD pipeline.

THE ONE-LINER THAT FRAMES EVERY ANSWER

"It's a FastAPI async backend where **fast things happen in the request** (auth, CRUD, cache reads) and **slow things happen out of band** (PDF embedding in a Celery worker, LLM calls released from the DB connection), fronted by Nginx and pooled through PgBouncer so it survives concurrency on cheap hardware."

The component map

Eight moving parts. Know what each one is and what would break if you removed it.

Component	Technology	Its one job	Remove it and...
Reverse proxy	Nginx	The only internet-facing process. TLS, rate limit, gzip, load-balance, buffer slow clients.	FastAPI workers get held hostage by slow clients; no edge throttling.
API	FastAPI + Uvicorn	Async HTTP layer: auth, validation, business logic orchestration.	No app.
Worker	Celery	Runs the slow PDF→embedding pipeline off the request path.	Uploads would block for 30–90s and time out.
Broker / cache	Redis	Celery queue, response cache, rate-limit counters, pub/sub for WebSockets.	No background jobs, no cache, no real-time push.
Pooler	PgBouncer	Multiplexes hundreds of app connections onto ~10 real Postgres connections.	Postgres hits its connection ceiling under load.
Database	PostgreSQL	Source of truth: users, documents, chats, quizzes, attempts, audit log.	No persistence.
Object store	MinIO / AWS S3	Stores the raw PDF bytes (binary blobs don't belong in Postgres).	Nowhere to put the files.
Vector DB	Pinecone	Stores chunk embeddings; does similarity search for RAG.	Chat can't find relevant context.
AI	Gemini (+ LangChain)	Embeddings model + the chat/quiz LLM.	No intelligence layer.

The request lifecycle — what happens on every single call

This is the spine. An interviewer who asks "walk me through a request" wants exactly this:

- 1 **Browser → Nginx :80/:443.** Nginx terminates TLS, applies an IP-level rate limit, and decides which `location` block matches.
- 2 **Nginx → Uvicorn :8000** via the `upstream` pool using `least_conn` load balancing over a keep-alive connection. Nginx injects `X-Forwarded-For` and a `X-Correlation-ID`.
- 3 **CorrelationIDMiddleware** runs first: reads/creates the correlation ID, stores it in a `ContextVar`, starts a latency timer, increments the Prometheus in-flight gauge.
- 4 **Routing + dependency injection.** FastAPI matches the route and resolves its `Depends()` tree: `get_current_user` (decode JWT → load User), `get_db` (lease an AsyncSession), and any rate-limit dependency (atomic Redis `INCR`).
- 5 **Pydantic validates** the request body against the schema. Invalid → automatic `422` before your code runs.
- 6 **Controller → Service → Repository.** The thin route handler calls a service (business logic), which calls repositories (SQL) and infrastructure clients (Redis/S3/Pinecone/Gemini).
- 7 **Response serialised** by the Pydantic `response_model`, gzipped by Nginx, returned. Middleware records latency + status into Prometheus and logs one structured JSON line with the correlation ID.
- 8 **BackgroundTasks fire after the response is sent** — e.g. the audit-log write — so they never add to the user's perceived latency.

The layered architecture (and why it's layered)

The backend is split into strict layers. This is the "**separation of concerns**" answer and it's worth stating explicitly because it shows intentional design:

```
# Request flows DOWN; each layer only knows the layer directly below it.
api/v1/           # Controllers – thin. Parse request, call a service, return. No SQL here.
|
services/         # Business logic – the "what". Orchestrates repos + infra. No FastAPI here.
|
repositories/     # Data access – the "how". Pure SQLAlchemy queries. One table family each.
|
models/           # ORM table definitions (SQLAlchemy declarative).
schemas/          # Pydantic request/response contracts (the API's public shape).
infrastructure/   # External clients: Redis, S3/MinIO, Pinecone – one connection helper each.
core/             # Cross-cutting: security, rate_limiter, circuit_breaker, logging, metrics, rbac.
```

INTERVIEW Q — "WHY SEPARATE SERVICES FROM REPOSITORIES? ISN'T THAT OVER-ENGINEERING?"

Because they change for *different reasons*. A repository changes when the **database schema** changes. A service changes when a **business rule** changes. Keeping them apart means I can swap Pinecone for pgvector by editing one infra file, unit-test business logic by mocking the repo, and read any endpoint in seconds because the controller is five lines. It also kills the classic "fat controller" anti-pattern where SQL, validation, and HTTP all tangle together.

The Tech Stack, Tool by Tool

For each tool: what it is, why it's here (vs the alternative), how DocuBrain actually uses it, and the questions you'll be asked.

2.1 FastAPI + Uvicorn — the async web layer

What it is. FastAPI is a Python web framework built on **ASGI** (the async successor to WSGI). **Uvicorn** is the ASGI server that actually runs the event loop and speaks HTTP. FastAPI gives you routing, dependency injection, and automatic request/response validation via Pydantic; Uvicorn gives you the async runtime.

Why here. DocuBrain is **I/O-bound**, not CPU-bound: almost every request waits on the database, Redis, S3, Pinecone, or Gemini. Async is the perfect fit — while one request is waiting on Postgres, the same OS thread serves another request. A traditional sync framework (Flask on Gunicorn) would block a whole worker thread on each wait.

THE MENTAL MODEL — ASYNC ≠ PARALLEL

There is **one thread, one event loop**. `async/await` is *cooperative multitasking*: at every `await`, the coroutine voluntarily yields control back to the loop, which runs other ready coroutines while the first one's I/O completes. It's a single chef juggling many pots, not many chefs. This is why a single Uvicorn worker handles thousands of concurrent *waiting* connections — but also why one `time.sleep()` or a heavy CPU loop **freezes the entire server** (it never yields).

How DocuBrain uses it.

- **Dependency injection** for everything cross-cutting: `Depends(get_current_user)`, `Depends(get_db)`, `Depends(general_rate_limit)`. Each route declares what it needs; FastAPI builds and tears down the tree per request.
- **Pydantic response models** (`response_model=DocumentResponse`) double as the serializer and as the OpenAPI/Swagger schema — the docs at `/docs` are generated, never hand-written.
- **The golden rule we follow:** any blocking SDK (the MinIO client, the Gemini sync SDK, pypdf) is pushed to a thread with `anyio.to_thread.run_sync(...)` so it never blocks the event loop.
- Single worker per container (`uvicorn --workers 1`) because we scale *horizontally* with more containers behind Nginx, not with in-process workers.

Q Why FastAPI over Flask/Django?

A I/O-bound workload + native async + automatic validation and OpenAPI docs from type hints. Django's ORM is sync-first and heavyweight for an API-only service; Flask would need a lot of bolt-ons (marshmallow, flask-restx) to match what FastAPI gives out of the box.

Q What is ASGI and how is it different from WSGI?

A WSGI is a *synchronous* contract: one request occupies one worker until it returns. ASGI is an *async* contract that also supports long-lived connections (WebSockets, SSE) — which WSGI fundamentally can't do. DocuBrain needs both async HTTP and WebSockets, so ASGI is mandatory.

Q What happens if you put a CPU-heavy loop in an async route?

A It blocks the event loop and stalls every other concurrent request on that worker. The fix is to offload to a thread (`run_in_executor` / `anyio.to_thread`) or, for truly heavy work, a separate process — which is exactly why PDF processing lives in Celery, not in the request.

2.2 PostgreSQL — the source of truth

What it is. A battle-tested relational database (ACID, SQL, strong typing, rich indexing, JSONB). DocuBrain runs Postgres 15.

Why here. The data is **relational** and **consistency matters**: users own documents, documents own quizzes, quizzes own items and attempts. Foreign keys with `ON DELETE CASCADE` enforce that graph at the DB level. We also lean on Postgres-specific features: **JSONB** columns (quiz options, answers, audit metadata), **DATE_TRUNC** for weekly analytics, and partial/composite indexes.

How DocuBrain uses it. Eight tables: `users`, `documents`, `chat_sessions`, `chat_messages`, `quizzes`, `quiz_items`, `quiz_attempts`, `audit_logs`. Every foreign key that points at a user/document is **indexed** (the access pattern is always "give me everything for this user"). The audit log uses `BigInteger` PK and `ON DELETE SET NULL` so records survive user deletion.

INDEXING — THE ANSWER THAT ALWAYS COMES UP

Every query in the repositories filters by `user_id` or a foreign key, and every one of those columns has an index. **Why it matters**: without an index, Postgres does a *sequential scan* — reads every row — which is $O(n)$ and gets linearly slower as data grows. A B-tree index makes it $O(\log n)$. The composite index `(user_id, created_at)` on chat sessions also serves the `ORDER BY created_at DESC` for free, because B-tree indexes are pre-sorted.

2.3 asyncpg vs psycopg2 — two drivers, on purpose

DocuBrain deliberately ships **two** Postgres drivers, and explaining why is a great signal of understanding the async/sync split:

Driver	Used by	Why
<code>asyncpg</code>	FastAPI (the API process)	Native async driver — non-blocking, integrates with the event loop. Fast.
<code>psycopg2</code>	Celery worker (a separate OS process)	The worker is plain synchronous Python with <i>no event loop</i> . A sync driver is the correct, simpler choice there.

The same `DATABASE_URL` is rewritten at startup: `postgresql://` → `postgresql+asyncpg://` for the API, and the plain `postgresql://` form for the worker. There's also normalisation of the legacy `postgres://` scheme (Heroku/Render) → `postgresql://`, since SQLAlchemy 2.0 rejects the old form.

THE PGBOUNCER GOTCHA WE HANDLE: `STATEMENT_CACHE_SIZE=0`

`asyncpg` caches prepared statements *per connection*. PgBouncer in **transaction mode** hands each transaction a possibly-different real connection, so a statement prepared on connection A doesn't exist on connection B → `"prepared statement does not exist"`. We disable `asyncpg`'s prepared-statement cache (`statement_cache_size=0`) so the app is safe behind a transaction-mode pooler. This is a senior-level detail — mention it.

2.4 SQLAlchemy 2.0 (async) — the ORM

What it is. The de-facto Python ORM. 2.0 introduced first-class async support via `AsyncSession` and `async_sessionmaker`.

How DocuBrain uses it.

- **Three engines:** a primary async engine (writes + read-your-writes), an optional **read-replica** async engine (analytics/progress), and a sync engine for Celery.
- **Right-sized pool:** `pool_size=5, max_overflow=10` (hard ceiling 15) — tuned for free-tier hosts where the managed Postgres pooler caps connections. `pool_pre_ping=True` silently recycles dead connections; `pool_recycle=300` drops connections after 5 min to survive NAT/DNS timeouts.
- `expire_on_commit=False` — so ORM objects stay usable *after* commit, which matters in async code where a lazy-load after commit would explode.
- **Eager loading with** `selectinload` to defeat the N+1 problem (see box).

INTERVIEW Q — "WHAT'S THE N+1 QUERY PROBLEM AND HOW DID YOU AVOID IT?"

N+1 = you fetch a list of N quizzes (1 query), then loop and lazily load each quiz's items (N more queries) = N+1 round-trips. In async SQLAlchemy a lazy load after the session context can also raise. DocuBrain uses `.options(selectinload(Quiz.items))` — SQLAlchemy fetches all parents, then loads *all* children in **one** extra `WHERE id IN ( ... )` query. Two queries total, regardless of N.

2.5 Alembic — schema migrations

What it is. SQLAlchemy's migration tool — version control for your database schema. Each change is a numbered, reversible script.

Why here. The schema must evolve **safely and identically** across dev, CI, and production. DocuBrain has four ordered migrations (`001_initial` → `002_audit_log` → `003_quiz_tables` → `004_quiz_attempts`). The container entrypoint runs `alembic upgrade head` *before* Uvicorn starts, so the schema is always current by the time the app serves traffic. CI runs the same command against a throwaway Postgres so a broken migration fails the build, not production.

WHY ALEMBIC MUST BYPASS PGBOUNCER

Migrations run multiple DDL statements that must share one persistent session. PgBouncer transaction mode drops the connection between transactions, which breaks that. So migrations connect **directly to Postgres** (`db:5432`), never through the pooler. This is called out explicitly in the stress-test script.

Q Why not just `Base.metadata.create_all()` ?

A `create_all` only creates tables that don't exist — it can't *alter* an existing table (add a column, change a type, add an index) without dropping data. Alembic produces explicit, reviewable, reversible diffs, which is the only safe way to change a production schema.

2.6 PgBouncer — the connection pooler ("the bouncer")

What it is. A lightweight connection pooler that sits between your app and Postgres. The app opens connections to PgBouncer (cheap); PgBouncer maintains a small pool of *real* Postgres connections and multiplexes everyone onto them.

WHY IT EXISTS — THE PROBLEM IT SOLVES

Every Postgres connection is a **separate OS process** on the server costing ~5–10 MB RAM. Postgres caps total connections (often 100; free-tier poolers cap at 15–20). With 1 API container × 15 pool + 1 worker × 3 + bursts, you blow past the ceiling and new connections get rejected. PgBouncer accepts up to **200 client connections** (`MAX_CLIENT_CONN`) but keeps only ~10 **real ones** open to Postgres (`DEFAULT_POOL_SIZE`). Hundreds of clients, a handful of real connections.

How DocuBrain configures it. `POOL_MODE=transaction` — a real connection is borrowed only for the duration of one transaction (`BEGIN...COMMIT`), then returned. This is the highest-throughput mode because connections are held for milliseconds, not for a whole client session. The app connects to `pgbouncer:5432` instead of `db:5432` — a one-line change in the connection string.

Pool mode	Connection held for...	Throughput	Caveat
session	the whole client session	low	safe — all features work
transaction ✓	one transaction	high	no session state (SET, LISTEN/NOTIFY, session prepared stmts)
statement	one statement	highest	breaks multi-statement transactions

Q What are the trade-offs of transaction mode?

A You lose session-scoped features: `SET` / `RESET` , `LISTEN` / `NOTIFY` , and session-level prepared statements. DocuBrain uses none of those in the request path — and it specifically disables asyncpg's prepared-statement cache (`statement_cache_size=0`) to be compatible. Migrations, which *do* need a persistent session, bypass PgBouncer and hit Postgres directly.

2.7 Redis — one tool, four jobs

What it is. An in-memory key-value store. Sub-millisecond reads/writes because everything lives in RAM. In DocuBrain it wears **four different hats** — calling this out shows you understand it's not "just a cache":

Role	What it does	Where
1 Cache	Caches document lists (30s TTL) and chat-session lists (60s TTL). Cache-aside pattern: check Redis → miss → hit Postgres → write back.	<code>document_service</code> , <code>chat_service</code>
2 Broker	The Celery message queue. The API pushes a task; the worker pops it.	<code>document_worker</code>
3 Rate limiter	Atomic <code>INCR</code> counters per <code>(user, time-window)</code> implement the fixed-window limit.	<code>core/rate_limiter</code>
4 Pub/Sub	The worker <code>PUBLISH</code> es "document ready" to a per-user channel; the WebSocket endpoint is <code>SUBSCRIBE</code> d and pushes it to the browser.	<code>ws.py</code> ↔ worker

WHY INCR IS THE RIGHT PRIMITIVE FOR RATE LIMITING

Redis is single-threaded, so `INCR` is **atomic** — even with 50 simultaneous requests, the counter increments cleanly with no race condition. The key is `rate:{window}:{user_id}:{bucket}` where `bucket = floor(now / window)`. First hit in a window arms a TTL of `2×window` so the key self-deletes — no cleanup job needed. The app separates broker traffic (`/0`) from cache traffic (`/1`) using different Redis logical databases.

Q Redis is in-memory — what happens on restart / what about durability?

A For DocuBrain's uses it's acceptable to lose Redis: cache entries just repopulate from Postgres, rate-limit windows reset (fail-open), and a lost queued task is re-dispatchable. The *source of truth* is always Postgres. If we needed durability Redis offers RDB snapshots and AOF (append-only file), but we don't rely on it here.

Q Fixed-window vs sliding-window rate limiting?

A We use fixed-window (simplest, one `INCR`). Its flaw is the boundary burst — a client can do 2× the limit across a window edge. Sliding-window log or token-bucket fixes that but costs more memory/complexity. For an app-level guard sitting behind Nginx's own IP limiter, fixed-window is the right trade-off.

2.8 Celery — the background worker ("the muscle")

What it is. A distributed task queue. You decorate a function with `@celery_app.task` ; calling `.delay(arg)` serialises the call into Redis and returns instantly. A separate worker process pops the message and runs the function.

WHY THE PDF PIPELINE MUST BE ASYNC/OUT-OF-BAND

Processing a PDF = download from S3 + extract text + chunk + call Gemini embeddings for every chunk (throttled ~10/s) + upsert to Pinecone. For a big document that's **30–90 seconds**. If that ran inside the upload request: the HTTP connection would time out, an Uvicorn worker would be blocked the whole time, and the user would stare at a spinner. Instead, upload returns in milliseconds with `202-style` "processing", and Celery does the heavy lifting. The user is notified via WebSocket when it's done.

How DocuBrain configures it (and why each setting matters):

```
celery_app.conf.update(
    broker_connection_retry_on_startup=True, # survive a slow-booting Redis
    task_acks_late=True, # ACK only AFTER the task finishes, not when picked up
    worker_prefetch_multiplier=1, # grab one task at a time - fair for long jobs
    task_track_started=True,
)
@celery_app.task(bind=True, max_retries=3, default_retry_delay=10)
def process_document_task(self, doc_id): ...
```

- `acks_late=True` is the big one: if the worker *crashes mid-task*, the message isn't lost — it's redelivered to another worker. This gives **at-least-once** delivery.
- `prefetch_multiplier=1`: by default a worker greedily reserves many messages. For long jobs that means one worker hoards the queue while others idle. Setting it to 1 = "only take what you can work on now."
- `max_retries=3` with exponential backoff, plus a *special* 60-second retry when Gemini returns `ResourceExhausted` (quota hit) so the per-minute quota window resets.

THE CONNECTION-DISCIPLINE PATTERN INSIDE THE WORKER (SENIOR-LEVEL)

The worker opens a DB session, reads the doc's metadata, marks it `processing`, and **closes the session before any slow I/O**. The 30–90s of S3+Gemini+Pinecone work happens holding *no* DB connection. Then a fresh short-lived session marks it `ready`. Why? A psycopg2 connection held idle for 90s while waiting on an external API will be killed by PgBouncer/Postgres timeouts, and the later commit would fail. "Acquire late, release early" keeps connections busy for milliseconds, not minutes.

Q At-least-once vs exactly-once delivery?

A Exactly-once is effectively impossible in a distributed queue (the classic two-generals problem). Celery with `acks_late` gives at-least-once: a task may run twice if a worker dies after doing work but before ACKing. The correct response is to make tasks **idempotent** — re-processing the same `doc_id` just re-embeds and upserts to the same Pinecone IDs (`{doc_id}_{i}`), overwriting cleanly. Same end state, no duplicates.

Q Why Redis as the broker and not RabbitMQ?

A Redis was already in the stack for caching, so reusing it as the broker removes a whole component. RabbitMQ is a more feature-complete broker (complex routing, guaranteed delivery semantics) but it's overkill here — we have one task type and one queue. Fewer moving parts wins.

2.9 Pinecone — the vector database

What it is. A managed vector database. Instead of rows, it stores **embeddings** — high-dimensional numeric vectors (DocuBrain uses **768 dimensions**) — and does fast **approximate nearest-neighbour** search by cosine similarity.

WHAT AN EMBEDDING ACTUALLY IS

An embedding model maps a piece of text to a point in 768-dimensional space such that **semantically similar text lands close together**. "How do I reset my password?" and "I forgot my login" end up near each other even though they share no words. The vector DB's job: given the query's vector, return the *k* stored chunk-vectors closest to it. That "closeness" is what makes retrieval *semantic*, not keyword-based.

How DocuBrain uses it. When a document is processed, each ~1500-char chunk becomes a vector with metadata `{text, source, doc_id, user_id}`, upserted with ID `{doc_id}_{i}`. At chat time the query is embedded and Pinecone is searched with `top_k=5` and a **metadata filter** `{user_id: ...}` — this is the multi-tenant isolation: user A can never retrieve user B's chunks. The index uses `metric="cosine"` and is created automatically on first boot if missing.

Q Why cosine similarity and not Euclidean distance?

A Cosine measures the *angle* between vectors, ignoring magnitude. For text embeddings, direction encodes meaning while magnitude is mostly noise (e.g. document length), so cosine is the standard choice. Two vectors pointing the same way score 1.0 regardless of length.

Q Could you replace Pinecone with something self-hosted?

A Yes — `pgvector` (a Postgres extension) would let me store vectors in the DB I already run, removing a managed dependency, at the cost of doing my own index tuning (IVFFlat/HNSW). Because all vector access is behind one infra client, it'd be a localized swap. Pinecone wins on zero-ops; pgvector wins on cost and data locality.

2.10 Google Gemini — embeddings + the LLM

What it is. Google's AI model family. DocuBrain uses it for **two distinct jobs**:

- **Embeddings** — `gemini-embedding-001` (768-dim) turns chunks and queries into vectors. Runs in the Celery worker (documents) and in the request (the query).
- **Generation** — `gemini-2.5-flash` is the chat LLM and the quiz/flashcard generator. "Flash" = the fast, cheap tier, which suits an interactive study tool.

Two clever uses worth mentioning:

- **Streaming chat** via LangChain's `model.astream(prompt)` — tokens are pushed to the browser as they're generated (Server-Sent Events), so the user sees the answer typing out instead of waiting for the whole thing.
- **Structured JSON output** for quizzes: `response_mime_type="application/json"` forces Gemini to return parseable JSON, not prose with markdown fences — so we can deserialize it directly into quiz items.

DEFENSIVE PARSING OF LLM OUTPUT

LLMs are non-deterministic, so the quiz parser *validates every item*: skip any question that doesn't have exactly 4 options, clamp an out-of-range `correct_answer` to 0, and if the whole response isn't valid JSON, return a clean `502` "AI returned an invalid response" rather than crashing. Never trust model output structurally.

2.11 LangChain — the LLM orchestration glue

What it is. A framework for composing LLM apps. DocuBrain uses it narrowly and deliberately: only

`ChatGoogleGenerativeAI` for the **streaming chat** integration, because LangChain's `.astream()` gives a clean async token iterator. The quiz path skips LangChain and calls the raw Gemini SDK directly (it needs the JSON-mode config). Using the right tool for each path — not forcing everything through one abstraction — is itself a good answer.

2.12 MinIO / AWS S3 — object storage ("the warehouse")

What it is. S3 is AWS's object store for blobs. **MinIO** is an open-source, S3-API-compatible server you run locally — so dev uses MinIO in Docker and prod uses real S3 with **zero code change** (same SDK, just a different endpoint).

WHY PDFS GO TO S3 AND NOT INTO POSTGRES

Databases are tuned for small, structured, frequently-queried rows — not multi-MB binary blobs. Storing files in Postgres bloats the DB, slows backups, wastes the buffer cache, and you pay DB pricing for dumb bytes. The pattern is universal: **blobs in object storage, a pointer (the S3 key) in the database**. DocuBrain stores `minio_path` on the `documents` row and the bytes in the bucket.

How DocuBrain uses it. Upload streams bytes straight into S3 via `io.BytesIO` (never touches local disk). The key is `{uuid}_{safe_filename}` to guarantee uniqueness. The MinIO SDK is synchronous, so every call is wrapped in `anyio.to_thread.run_sync` and guarded by the S3 **circuit breaker**. Bucket auto-creates on startup (`ensure_bucket`).

2.13 Nginx — the reverse proxy ("the gatekeeper")

What it is. A high-performance, event-driven web server / reverse proxy. It's the **only** container that exposes ports to the internet — everything else lives on Docker's internal network. That alone is the right security posture.

The eight jobs DocuBrain's Nginx does (memorise these — it's a whole interview topic by itself):

1. **TLS termination** — decrypt HTTPS once at the edge; plain HTTP inside the trusted Docker network.
2. **Rate limiting** — a coarse per-IP limit (`limit_req_zone`) that rejects floods *before* Python ever wakes up. A `geo` block exempts localhost/Docker-bridge so load tests aren't throttled as one IP.
3. **Load balancing** — `upstream` pool with `least_conn` (route to the worker with the fewest active connections — correct when endpoint latencies vary wildly from 1ms to 5s).
4. **Response buffering** — Nginx reads the full response from FastAPI fast, frees the worker, then dribbles it out to a slow client itself. Slow clients can't tie up Python workers.
5. **Gzip** — compresses JSON ~70–80% before it hits the wire.
6. **Real-IP forwarding** — injects `X-Forwarded-For` so the app sees the true client IP through the proxy chain.
7. **WebSocket upgrade** — explicitly forwards the `Upgrade` / `Connection` headers so the WS handshake (HTTP 101) succeeds, with a 1-hour read timeout for long-lived connections.
8. **Endpoint hardening** — blocks `/metrics` from the public internet (403), exempts `/health` from rate limits, and adds security headers (`X-Content-Type-Options: nosniff` , `X-Frame-Options: DENY` , etc.).

WHY `LEAST_CONN` BEATS ROUND-ROBIN HERE

Round-robin sends request N to worker (N mod k) blindly. But DocuBrain's endpoints range from `/health` (~1ms) to `/quiz/generate` (~5s). Round-robin would keep piling requests on a worker already stuck on three quiz generations. `least_conn` sees the queued connections and routes around the busy worker — much better tail latency under mixed load.

SSE/STREAMING + BUFFERING = THE GOTCHA

Buffering is great for normal JSON but **breaks streaming** — for chat SSE you want each token flushed immediately, not buffered. The chat endpoint sets `X-Accel-Buffering: no` in its response headers to tell Nginx "don't buffer this one." Knowing this exception is a strong signal.

Q Why is Nginx event-driven and why does that matter?

A Each Nginx worker uses Linux `epoll` to watch thousands of sockets in one process — O(1) notification when a socket is ready, instead of one thread per connection. That's why a single Nginx box holds 10k+ idle connections cheaply, which is exactly what you want fronting slow mobile clients.

2.14 Docker & docker-compose — packaging and orchestration

What it is. Docker packages the app + its exact dependencies into an image that runs identically everywhere. docker-compose declares the whole multi-container stack in one YAML file.

Strong choices DocuBrain makes:

- **Multi-stage build** — a `builder` stage compiles dependencies (needs gcc, libpq-dev); the `runtime` stage copies only the installed packages, so the shipped image has no compilers. Smaller image, smaller attack surface.

- **Non-root user** — the container runs as `appuser`, not root. If the app is compromised, the blast radius is limited.
- **Healthchecks** + `depends_on: condition: service_healthy` — the backend only starts after Postgres, Redis, and MinIO report healthy. No race-condition crashes on boot.
- **Layer caching** — `requirements.txt` is copied and installed *before* the app code, so a code change doesn't bust the dependency layer. Faster rebuilds.

Q Container vs VM?

A A VM virtualises hardware and ships a whole guest OS (GBs, slow boot). A container shares the host kernel and isolates only the process/filesystem/network namespaces (MBs, boots in ms). For packaging a stateless app service, containers are far lighter — you run many per host.

Q Why multi-stage builds?

A To keep build-time tools (gcc, dev headers) out of the runtime image. Smaller image = faster pulls/deloys and a smaller security surface — no compiler for an attacker to use.

2.15 The auth stack — JWT, passlib/bcrypt, python-jose

DocuBrain implements a modern **two-token** auth scheme. The pieces:

Library	Job
<code>passlib</code> + <code>bcrypt</code>	Hash passwords. bcrypt is a deliberately <i>slow</i> , salted hash — brute-forcing stolen hashes is computationally expensive.
<code>python-jose</code>	Encode/decode JWTs (HS256).
<code>python-multipart</code>	Parse the OAuth2 form-login body and file uploads.

THE TWO-TOKEN PATTERN — THE CENTREPIECE SECURITY ANSWER

Access token: a short-lived (15-min) JWT returned in the response body, sent as `Authorization: Bearer`. Stateless — the server verifies the signature, no DB lookup needed.

Refresh token: a long-lived (7-day) cryptographically-random string (*not* a JWT) stored in Redis and delivered as an **httpOnly cookie** (JavaScript can't read it → immune to XSS token theft). When the access token expires, the browser silently calls `/auth/refresh`; the cookie is sent automatically, Redis is checked, and a fresh pair is issued.

Rotation = theft detection: each refresh *deletes* the old token from Redis before issuing a new one. If a stolen token is replayed after the real user already rotated, the lookup misses → 401. The breach is detected.

Q Why is the access token a JWT but the refresh token isn't?

A The access token is verified on *every* request — making it a self-contained, signature-verifiable JWT avoids a DB hit each time (stateless). The refresh token is used rarely (every 15 min) and *must be revocable*, so it's a random opaque key looked up in Redis — deleting the key instantly kills the session. You can't revoke a pure JWT without extra infrastructure; that's the whole point of the split.

Q Where do you store tokens client-side, and why?

A Refresh token in an **httpOnly, Secure, SameSite cookie** — unreadable by JS, so XSS can't steal it. The short-lived access token lives in memory/JS; even if leaked it expires in 15 minutes. This splits the XSS risk (cookie) from the CSRF

risk (bearer header) instead of being vulnerable to both.

Q What's actually inside the JWT?

A Header (alg=HS256), payload claims (`sub` =email, `user_id` , `exp`), and an HMAC signature over the first two using the server's `SECRET_KEY` . Anyone can *read* the payload (it's just base64) — so never put secrets in it — but nobody can forge the signature without the key. `get_current_user` decodes it, then loads the live `User` from the DB by `user_id` .

2.16 Prometheus — metrics

What it is. A pull-based metrics system. Your app exposes `/metrics` in a text format; Prometheus scrapes it on an interval and stores time-series data you can graph/alert on.

How DocuBrain uses it. The middleware records three metric types on every request: a **Counter** (`http_requests_total` by method/path/status), a **Histogram** (request latency, bucketed 50ms→10s to cover both fast CRUD and slow AI calls), and a **Gauge** (requests in flight). Crucially, the label uses the *route template* (`/documents/{doc_id}`) not the real path (`/documents/42`) — otherwise every ID becomes a new time series and you get "cardinality explosion."

Q Counter vs Gauge vs Histogram?

A Counter only goes up (total requests). **Gauge** goes up and down (in-flight requests, memory). **Histogram** buckets observations so you can compute percentiles (p95 latency) server-side. Pick by what the number does over time.

Q Push vs pull metrics — why is Prometheus pull?

A Pull means Prometheus controls scrape timing and can detect a target that's *down* (scrape fails) — a push model can't tell "healthy but quiet" from "dead." Pull also avoids every instance needing to know the metrics backend's address. For ephemeral jobs that die before a scrape, Prometheus offers a Pushgateway as the exception.

2.17 Terraform — infrastructure as code

What it is. A declarative tool to define cloud infrastructure in code. You describe the *desired* state (a VPC, a subnet, an EC2 box, an S3 bucket, IAM roles); Terraform diffs it against reality and makes the minimum changes to converge.

What DocuBrain provisions. A complete AWS footprint: a **VPC** (isolated network) → **Internet Gateway** → public **subnet** → **route table** → **security groups** (firewall) → an **EC2** instance (gp3 encrypted disk) with a `user_data` boot script that installs Docker and launches the stack → an **Elastic IP** (stable address for DNS) → an **S3 bucket** + **IAM instance profile** so the box can reach S3 without hard-coded keys.

WHY IAC INSTEAD OF CLICKING IN THE AWS CONSOLE

The infrastructure is **reproducible, reviewable, and version-controlled**. You can destroy and recreate the entire environment identically, code-review a network change in a PR, and never suffer "it works because someone clicked something nobody documented." `default_tags` stamp every resource for cost tracking; `prevent_destroy` guards the prod server from an accidental wipe.

Q Terraform state — what is it and why does it matter?

A Terraform records what it created in a *state file* that maps your code to real resource IDs. For a team you store it remotely (S3 + a DynamoDB lock table) so two people can't apply simultaneously and corrupt it. Locally it's a local file — fine solo, dangerous in a team.

Q Declarative vs imperative infra?

A Imperative (a bash script) says "do these steps." Declarative (Terraform) says "this is the end state I want" and lets the tool compute the diff. Declarative is idempotent — running it twice changes nothing the second time — which is what you want for infrastructure.

2.18 GitHub Actions — the CI/CD pipeline

What it is. GitHub's built-in automation. On every push/PR it spins up fresh Ubuntu runners and executes your pipeline. DocuBrain's pipeline is a classic sequential gate:

- 1 **Lint** (Ruff) — cheapest check first; catches typos/unused imports in ~5s before wasting time on tests.
- 2 **Test** (pytest) — runs against *real* Postgres + Redis **service containers**, after applying `alembic upgrade head` (so migrations themselves are tested). Only runs if lint passed.
- 3 **Build** — multi-stage Docker image, tagged with the git SHA and `latest`, pushed to a registry with layer cache. Only on push to `main`, never on PRs.
- 4 **Deploy** — SSH to the server, pull the new image, stop-old/start-new swap, then a health check that rolls back on failure.
- 5 **Post-deploy smoke test** — k6 hits the *live* production URL to catch the bugs that only appear in prod (wrong env var, missing secret).

TWO IDEAS INTERVIEWERS LOVE HERE

Tag images with the git SHA, not just `latest` — that's what makes rollbacks deterministic ("redploy `:a3f9d2c`"). And **the SHA tag is how deploy knows exactly what shipped**. Second: **fail fast / cheap-checks-first** ordering — lint (5s) before test (2min) before build before deploy. You never burn 2 minutes of CI to fail on a missing comma.

Q Difference between CI and CD?

A **CI** (Continuous Integration) = automatically lint/test/build every change so `main` is always known-good. **CD** (Continuous Delivery/Deployment) = automatically ship that known-good artifact to production. DocuBrain does both: the first three jobs are CI, the last two are CD.

Q Why test against real Postgres/Redis containers instead of mocks?

A Mocks test that you called a function, not that the SQL is valid or the migration applies. Service containers give a real DB so tests catch genuine integration bugs — a malformed query, a broken migration — for a few seconds of extra runtime.

2.19 k6 — load & performance testing

What it is. A modern load-testing tool: you write the test in JavaScript, it runs many **virtual users (VUs)** concurrently from a single fast Go binary, and asserts on **thresholds** (e.g. "p95 latency < 200ms, error rate < 1%"). Breach a threshold → non-zero exit → CI fails.

How DocuBrain uses it. Three suites: **smoke** (1 VU, 30s sanity), **load** (ramp to 50 VUs over 5 min — the headline benchmark), and **spike** (200 VUs to deliberately trip circuit breakers and watch recovery). The banked load-test result:

THE NUMBERS TO QUOTE

50 concurrent VUs · p95 latency 31 ms · 0% error rate · 26.5 req/s sustained. Run on the full Dockerised stack (Nginx → FastAPI → PgBouncer → Postgres + Redis). The point isn't the raw throughput on a laptop — it's that the system stayed correct and fast under sustained concurrency with zero errors.

Q What's a percentile and why p95/p99 instead of average?

A Averages hide tail pain. If 95 requests are 20ms and 5 are 5s, the average looks fine but 1-in-20 users has a terrible time. p95 = "95% of requests were faster than this." Tail latency is what users actually feel, so SLOs are written on p95/p99, not the mean.

Q What does a spike test prove that a load test doesn't?

A Load tests verify steady-state. Spike tests verify *resilience* under sudden overload: do rate limiters return clean 429s? Do circuit breakers trip and fail fast instead of cascading? Does the system *recover* when the spike ends? It tests the failure machinery, not the happy path.

Endpoint-by-Endpoint Deep Flows

For each route: the exact path data takes, the design decisions baked in, and the gotchas already handled.

3.1 Auth — /auth/*

POST /auth/register

- 1 Pydantic `UserCreate` validates `email` (real `EmailStr` check) and `password` (8–128 chars). Bad input → 422 before any code runs.
- 2 `auth_service.register_user` checks the email isn't taken (`get_by_email`) → 400 if it is.
- 3 Password is hashed with **bcrypt** (salted, slow) — the plaintext is never stored.
- 4 User row is created; an **access JWT** (15-min) and a **random refresh token** (7-day) are minted. The refresh token is stored in Redis keyed by its value.
- 5 Refresh token is set as an **httpOnly cookie** scoped to `/auth` ; access token returns in the JSON body.
- 6 A `BackgroundTask` writes an audit-log entry *after* the response is sent.

POST /auth/login

Uses FastAPI's `OAuth2PasswordRequestForm` (form-encoded `username` / `password` , the OAuth2 standard). Looks up the user, verifies the password with `bcrypt.verify` , and — critically — returns the **same generic 401 "Incorrect email or password"** whether the email doesn't exist or the password is wrong.

WHY THE ERROR MESSAGE IS DELIBERATELY VAGUE

Distinct messages ("no such user" vs "wrong password") leak which emails are registered — a **user-enumeration** vulnerability an attacker uses to build target lists. One generic message closes it.

POST /auth/refresh & POST /auth/logout

Refresh: reads the httpOnly cookie (no JS involved), atomically pops the old token from Redis (read-then-delete = rotation), validates the user still exists, and issues a fresh pair. Replaying a rotated token → cache miss → 401 (theft detected). **Logout:** deletes the refresh token from Redis (instant server-side revocation) and clears the cookie. The access token remains valid until its ≤15-min expiry — an accepted trade-off of stateless tokens.

3.2 Documents — /upload, /documents

POST /upload/ — the most defensive endpoint

- 1 **Rate-limit gate first:** `upload_rate_limit` (10/hour) runs as a dependency *before* any I/O — expensive AI processing is protected up front.
- 2 Validate filename (present, ≤255 chars).

- 3 Read bytes; reject if >20 MB or empty.
- 4 **Magic-byte check:** `python-magic` inspects the actual file content (`application/pdf` ?) — not the file extension, which is trivially spoofed. A `.pdf` that's really an executable is rejected.
- 5 Stream bytes to S3 under key `{uuid}_{name}` , guarded by the **S3 circuit breaker** (→ 503 if storage is down).
- 6 Insert the `documents` row with `status="pending"` .
- 7 `process_document_task.delay(doc.id)` — hand off to Celery and return immediately. Invalidate the doc-list cache. Audit in the background.

THE KEY INSIGHT: UPLOAD RETURNS IN MILLISECONDS

The user gets an instant response with the document marked "pending/processing." All the slow work (embedding) happens in the worker. The browser learns it's done via a WebSocket push. This is the **async-handoff pattern** and it's the single most important flow in the app.

GET /documents/ & DELETE /documents/{id}

List: cache-aside — check Redis (30s TTL), miss → query Postgres ordered by upload date, write back. Only the default page (skip=0, limit=100) is cached. **Delete:** a true *cascade purge* — removes the S3 object, deletes all the document's vectors from Pinecone (`filter={doc_id}`), deletes the DB row, and invalidates the cache. Ownership is checked first (`doc.user_id == current_user.id`) → 404 if not yours (404 not 403, so you can't even confirm the doc exists).

3.3 The Celery worker pipeline — `process_document_task`

This is where the heavy lifting lives. Four phases with strict connection discipline:

- 1 **Short DB session:** read `minio_path` , `filename` , `user_id` ; set `status="processing"` ; commit; **close the session.**
- 2 **Slow I/O, no DB held:** download PDF from S3 → extract text with `pypdf` → chunk (1500 chars, 100 overlap) → embed every chunk via Gemini (throttled ~10/s) → upsert vectors to Pinecone in batches of 100.
- 3 **Fresh short session:** set `status="ready"` + a 200-char summary.
- 4 **Notify:** `PUBLISH` a `document.ready` message to Redis channel `doc:ready:{user_id}` → the WebSocket layer pushes it to the browser.

FAILURE HANDLING BAKED IN

Gemini quota error (`ResourceExhausted`) → retry after **60s** (lets the per-minute window reset), keep status as "processing." Any other error → set "failed" and retry (max 3, exponential backoff). Because re-running uses the same deterministic Pinecone IDs, retries are **idempotent** — no duplicate vectors.

WHY CHUNK + OVERLAP?

LLMs and embedding models have a context limit, and retrieval is sharper on small focused passages than on a whole document. **1500-char chunks** keep each vector semantically tight; the **100-char overlap** stops a sentence that straddles a chunk boundary from losing its meaning in both halves.

3.4 Chat — /chat/* and the streaming RAG flow

Sessions are simple CRUD (create / list / get messages / delete), with the session list cached in Redis for 60s. The interesting one is the stream:

POST /chat/sessions/{id}/stream — RAG over SSE

- 1 Verify the session belongs to the user. If it's still titled "New Chat", set its title to the first message.
- 2 Persist the user's message; load the last 10 messages as conversation history.
- 3 `await db.commit()` — **release the DB connection back to the pool BEFORE streaming.** The 10–30s LLM stream holds no DB connection.
- 4 **RAG:** embed the query (Gemini) → Pinecone `top_k=5` search filtered by `user_id` → join the retrieved chunks into a context block.
- 5 Build the prompt (context + history + question) and `astream` Gemini tokens, yielding each as an SSE `data:` frame to the browser in real time.
- 6 After the stream, append a `[SOURCES]` event (which documents were cited), then `[DONE]`. A fresh DB connection persists the full assistant reply.

THE TWO SENIOR-LEVEL DETAILS IN THIS ONE ENDPOINT

(1) **Commit-before-slow-IO:** an `AsyncSession` holds an `asyncpg` connection from its first query until `commit/close`. If you held it through a 30s LLM stream, that connection is dead weight — and `asyncpg` will close it idle, making the later write fail. Committing first returns it to the pool; the session transparently leases a fresh one for the final insert.

(2) **Every external call is wrapped in a circuit breaker.** Gemini and Pinecone each have one — if Gemini is down, the breaker trips and users get a fast, clean "temporarily unavailable" instead of 30s timeouts piling up.

SSE VS WEBSOCKET — WHY SSE FOR CHAT?

Chat streaming is **one-directional** (server → client, token by token). Server-Sent Events is exactly that: a long-lived HTTP response that flushes chunks, with auto-reconnect built into the browser's `EventSource`. It's simpler than a full-duplex WebSocket and rides normal HTTP/Nginx. We reserve WebSockets for the genuinely push-style notification channel.

3.5 Quiz — /quiz/*

POST /quiz/generate

- 1 Validate ownership + status is `ready` (else 400 "still processing").
- 2 **Capture `minio_path` as a string, then `await db.commit()`** — same release-the-connection pattern before the slow path.
- 3 Download PDF (S3 breaker) → extract text → cap at 500k chars (~125k tokens, within Gemini Flash's 1M window).
- 4 Build a quiz or flashcard prompt; call Gemini in **JSON mode** (Gemini breaker).
- 5 **Defensively parse:** drop malformed items, clamp bad indices, 502 on invalid JSON.

- 6 Persist quiz + items in one transaction; `selectinload` the items for the response.

POST /quiz/{id}/attempt — scoring

Answers arrive as a **flat ordered list** of 0-based option indices (`null` = skipped). The service sorts items by `order_index`, zips them with the answers, counts matches against `correct_answer`, stores the attempt, and returns a per-question **breakdown** (your answer, correct answer, explanation). Flashcard "attempts" store `score=null` — a review has no right/wrong.

ONE TABLE, TWO CONTENT TYPES (A CLEAN SCHEMA CHOICE)

`quiz_items` serves both quizzes and flashcards via nullable columns: a quiz item fills `content_front` + `options` + `correct_answer`; a flashcard fills `content_front` + `content_back` and leaves the quiz columns null. One table, one set of queries, no awkward inheritance.

3.6 Progress & Analytics — read-replica endpoints

Both `GET /progress/stats` and `GET /analytics/dashboard` use `Depends(get_read_db)` — they route to the **read replica** because a few ms of replication lag is fine for historical aggregates. Analytics runs five SQL aggregate queries (counts + a `DATE_TRUNC` weekly-trend + a top-documents join) — **all aggregation is pushed to Postgres**, never fetched row-by-row into Python. The queries run sequentially (an `AsyncSession` wraps one connection, so concurrent queries on it would deadlock — a subtle correctness note).

3.7 WebSocket — /ws/notifications

- 1 **Auth before accept:** the JWT arrives as a `?token=` query param (browsers can't set headers on WS). Invalid → close with policy-violation, never opening the socket.
- 2 Accept, then `SUBSCRIBE` to Redis channel `doc:ready:{user_id}`.
- 3 Run **two concurrent tasks**: one forwards Redis messages → browser; one reads incoming frames to detect disconnect.
- 4 `asyncio.wait(FIRST_COMPLETED)` — whichever finishes first (client leaves or Redis dies) tears down the other and cleans up the subscription. No zombie connections.

HOW THE WORKER REACHES THE BROWSER WITHOUT SHARING MEMORY

The Celery worker and the API are **separate processes** — they can't call each other directly. Redis **Pub/Sub** is the bridge: the worker `PUBLISH` es to a per-user channel; whichever API process holds that user's WebSocket is `SUBSCRIBE` d and relays the message. This is the decoupled, horizontally-scalable way to do real-time fan-out.

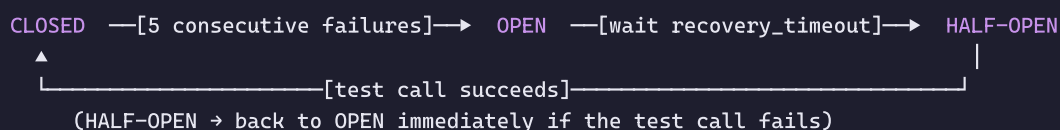
Cross-Cutting Patterns

These are the patterns that separate "I built a CRUD app" from "I built a resilient backend." Each is a likely deep-dive.

4.1 The Circuit Breaker

The problem. When an external dependency (Gemini, Pinecone, S3) goes slow or down, naive code keeps calling it. Every call waits the full timeout, threads pile up, and the failure *cascades* back through your system. One sick dependency takes down the whole app.

The pattern. A state machine that "fails fast" once a dependency looks broken:



- **CLOSED** — healthy; calls pass through, failures are counted.
- **OPEN** — tripped; calls are rejected *instantly* with `CircuitOpenError` → the service returns a clean **503**. No network call, no waiting. The dependency gets breathing room to recover.
- **HALF-OPEN** — after the cooldown, one probe call is allowed. Success → CLOSED (recovered). Failure → straight back to OPEN.

DocuBrain runs **three independent breakers**, tuned per service: Gemini (threshold 5, 60s recovery — AI APIs recover slowly), Pinecone (5, 30s), S3 (3, 20s — storage errors are usually infra-level, fail faster).

THE CONCURRENCY DETAIL IN THE IMPLEMENTATION

The breaker holds an `asyncio.Lock` only while reading/writing its *state* — never across the actual network call. Holding the lock during the call would serialize every request through that breaker and destroy async concurrency. Lock for the bookkeeping, release before the await. That's the kind of detail that signals you actually wrote it.

4.2 Two-Tier Rate Limiting

DocuBrain throttles at **two layers**, and they defend against different things:

Layer	Keyed by	Defends against	Runs...
Nginx (<code>limit_req_zone</code>)	client IP	volumetric floods / DDoS from one IP	at the edge, before Python wakes
App / Redis (<code>INCR</code>)	authenticated user_id	one logged-in user abusing expensive endpoints	as a FastAPI dependency

The app tier has two limits: **100 req/min** general, and a strict **10 uploads/hour** on the expensive AI path. Why both tiers? Nginx doesn't know *who* a request is (no JWT decode at the edge); the app doesn't see floods cheaply (they'd still spin up a request). IP-level at the edge + user-level in the app = defense in depth.

4.3 RBAC — a centralised policy layer

Authorization isn't scattered as `if user.role == "admin"` checks across the codebase — those rot and drift. Instead there's a single **policy map** in `core/rbac.py`: `role → set of permissions` (`"documents:read_own"` , `"chat:write"` , ...). Enforcement is a FastAPI dependency factory, `require_permission(" ... ")` , that raises 403 if the user's role lacks the permission.

WHY THIS DESIGN IS THE "RIGHT" RBAC ANSWER

One source of truth. To add a role or change what it can do, you edit one dictionary — every enforcement point updates automatically. Permissions are named `resource:action` , the industry convention. Adding authorization to a new endpoint is one line: `Depends(require_permission("documents:upload"))` . This is the difference between RBAC-as-a-pattern and authorization-as-spaghetti.

4.4 The Append-Only Audit Log

A dedicated `audit_logs` table records security-relevant actions (login, upload, delete, chat query) with `user_id` , action, resource id, JSONB metadata, IP, and timestamp. Two deliberate design choices:

- **Write-only by design** — the service exposes *only* a `log()` function. No update, no delete. The log is immutable, which is what makes it trustworthy for forensics. `ON DELETE SET NULL` means audit rows survive even after the user is deleted.
- **Written in a BackgroundTask** — it runs *after* the response is sent, on its own DB session, and **any failure is swallowed** (logged, never raised). A slow or failing audit write must never affect the user's request or its latency.

4.5 Observability — the three pillars, all present

Pillar	In DocuBrain	Answers the question...
Metrics	Prometheus counter/histogram/gauge via middleware	"Is it healthy <i>right now</i> ? What's p95?"
Logs	Structured single-line JSON, one per request	"What exactly happened on this request?"
Traces	Correlation IDs propagated end-to-end (the foundation for full tracing)	"Follow one request across every component."

CORRELATION IDS + CONTEXTVAR — THE ELEGANT BIT

Middleware reads or generates an `X-Correlation-ID` and stores it in a Python **ContextVar** — the async-safe equivalent of thread-local storage. Every log line emitted anywhere in that request's async call-tree automatically picks up the same ID via the JSON formatter, with zero code threading it through function arguments. The ID is echoed back in the response header so a client (or Nginx, or a gateway) can correlate end-to-end. This is the seed of distributed tracing.

4.6 The Read Replica & Eventual Consistency

DocuBrain supports an optional read replica: heavy read-only endpoints (analytics, progress) use `get_read_db()` , which routes to a replica engine. If `READ_REPLICA_URL` isn't set, it transparently falls back to the primary — so the same code runs in dev (one DB) and prod (primary + replica) with no branching.

THE CONSISTENCY RULE YOU MUST ARTICULATE

Replicas lag the primary by milliseconds (asynchronous replication). So the rule: **read-your-writes queries stay on the primary** (e.g. listing documents right after an upload — you must see it), while **staleness-tolerant aggregates go to the replica** (a dashboard that's 50ms behind is fine). Knowing *which* reads can tolerate lag is the whole skill — it's a direct application of eventual consistency.

4.7 The RAG Pipeline, End to End

Tying the AI flow together in one place, because "explain your RAG pipeline" is the headline AI question:

- 1 **Ingest (offline, in the worker):** PDF → text → 1500-char overlapping chunks → Gemini embeddings (768-dim) → upsert to Pinecone with `{text, doc_id, user_id}` metadata.
- 2 **Retrieve (online, per query):** embed the user's question → Pinecone cosine search `top_k=5`, filtered by `user_id` for tenant isolation.
- 3 **Augment:** stuff the retrieved chunks + recent chat history into a structured prompt that instructs the model to answer *only* from the provided context.
- 4 **Generate:** stream Gemini's answer token-by-token over SSE, then report which source documents were used.

WHY RAG INSTEAD OF FINE-TUNING?

Fine-tuning bakes knowledge into model weights — expensive, slow to update, and it can't cite sources. RAG keeps knowledge in an external store you can update instantly (upload a new PDF and it's searchable in seconds), grounds answers in real passages to reduce hallucination, and lets the model **cite** what it used. For a per-user document tool, RAG is the only sensible choice — you can't fine-tune a model per user.

The Roadmap — What Was Built & Why

DocuBrain was hardened in phases, each one a recognised production concern. This is the "show me you understand the journey from prototype to production" narrative.

Phase 0 — Foundations & Hardening Complete

- **Removed dead weight** (a legacy Jitsi integration) and slimmed dependencies from ~186 → 17 direct packages after an audit. Fewer deps = smaller attack surface and faster builds.
- **Secrets out of code** — every credential moved to environment variables, validated at startup (the app crashes loudly if a required secret is missing, rather than failing mysteriously later).
- **Alembic migrations** (4 of them) — schema is now versioned and reproducible.
- **CORS allowlist** — explicit origin list instead of `*`.
- **Pydantic hardening** — strict request schemas (length bounds, `EmailStr`, `Literal` enums) so bad input is rejected at the boundary.

Phase 1 — Security & Access Control Complete

- **Two-token auth** — 15-min access JWT + 7-day httpOnly refresh token with **rotation** (theft detection).
- **Two-tier rate limiting** — Nginx per-IP + Redis per-user.
- **RBAC policy layer** — centralised role→permission map enforced via DI.
- **Append-only audit log** with correlation IDs threaded through every request.

Phase 2 — Product Features Complete

- **AI quizzes & flashcards** (Gemini JSON mode) with attempt scoring and per-question breakdowns.
- **Progress tracking** — averages, best scores, documents studied, recent attempts.
- **Real-time notifications** — WebSocket + Redis Pub/Sub for "document ready."
- **Analytics dashboard** — weekly score trend (DATE_TRUNC) + most-studied documents, all as SQL aggregates.

Phase 3 — Observability Partial

- Done Structured JSON logging with correlation IDs.
- Done Prometheus metrics (counter / histogram / gauge) with route-template labels to avoid cardinality explosion.
- Open Sentry (error tracking) and OpenTelemetry (full distributed tracing) — the correlation-ID groundwork is already in place to adopt OTel cleanly.

HOW TO TALK ABOUT THE GAPS (TURN THEM INTO STRENGTHS)

"Observability is layered. I have metrics and structured logs with correlation IDs — the two pillars you need to operate the system. The natural next step is wiring those correlation IDs into OpenTelemetry spans for full distributed tracing and adding Sentry for exception aggregation. I deliberately built the correlation-ID plumbing first *because* it's the foundation tracing builds on." Naming the next step precisely shows maturity.

Phase 4 — Scaling the Data Tier Complete

- **PgBouncer** in transaction mode — hundreds of clients onto ~10 real connections.
- **Nginx** production config (TLS-ready, WebSocket upgrade, gzip, security headers, per-IP geo rate limiting, least-conn load balancing, response buffering).
- **Read-replica** infrastructure — heavy reads routed off the primary, with transparent fallback.

Phase 5 — Infrastructure & Delivery Complete

- **Terraform** — full AWS footprint (VPC, subnet, IGW, route table, security groups, EC2, Elastic IP, S3, IAM) as code.
- **GitHub Actions** — Lint → Test → Build → Deploy → post-deploy Smoke, with SHA-tagged images and rollback-on-health-failure.
- **k6** — smoke / load / spike suites; benchmark banked at **50 VUs, p95 31ms, 0% errors**.

THE ROADMAP AS A SINGLE SENTENCE

"I took a working AI prototype and walked it through the production checklist: **harden the foundations, lock down security, ship the features, make it observable, scale the data tier, and automate delivery**. Two items remain — Sentry and OpenTelemetry — and the logging/correlation-ID work I already did is the foundation for both."

The Interview Q&A Bank

100+ questions grouped by theme. Each answer is tuned to be said out loud in 20–40 seconds and to tie back to something concrete in DocuBrain.

6.1 Python, Async & Concurrency

Q Explain `async/await` like I'm a teammate.

A It's cooperative multitasking on one thread. `async def` makes a coroutine; `await` is a point where it pauses and hands control back to the event loop, which runs other ready coroutines while this one's I/O completes. Great for I/O-bound work, useless for CPU-bound work.

Q What is the event loop?

A A single-threaded scheduler that keeps a queue of ready tasks. It runs one until it hits an `await` on pending I/O, parks it, runs the next, and resumes parked tasks when their I/O is done. Uvicorn runs one per worker process.

Q What is the GIL and does it matter here?

A The Global Interpreter Lock lets only one thread execute Python bytecode at a time, so threads don't give true CPU parallelism. For DocuBrain it barely matters — the work is I/O-bound (waiting on DB/network), and `async` sidesteps the GIL by not needing threads at all. For CPU parallelism you'd use multiple *processes* (which is exactly what Celery workers are).

Q Concurrency vs parallelism?

A Concurrency = dealing with many things by interleaving (one `async` worker juggling 1000 connections). Parallelism = doing many things at literally the same instant (multiple CPU cores/processes). `Async` gives concurrency; Celery workers + multiple containers give parallelism.

Q How do you call a blocking/sync library from `async` code?

A Offload it to a thread pool so it doesn't block the loop: `anyio.to_thread.run_sync(fn)` or `loop.run_in_executor`. DocuBrain does this for every sync SDK — MinIO, pypdf, the sync Gemini client.

Q Threads vs processes vs `async` — when each?

A `Async` for high-concurrency I/O (the API). Threads for blocking I/O in libraries that aren't `async`-aware (wrap the sync SDK). Processes for CPU-bound or isolation needs (Celery workers). DocuBrain uses all three deliberately.

Q What's a `ContextVar`?

A `Async`-safe thread-local-style storage. Each request's `async` call-tree sees its own value. DocuBrain stores the correlation ID in one so every log line in a request auto-tags itself without passing the ID around.

6.2 Databases, ORM, Pooling & Transactions

Q What is connection pooling and why does it matter?

A Opening a DB connection is expensive (TCP + auth + a server-side process). A pool keeps a set of warm connections and lends them out. DocuBrain pools twice: SQLAlchemy's app-side pool (5+10) and PgBouncer in front of Postgres, which multiplexes 200 client conns onto ~10 real ones.

Q Walk me through ACID.

A Atomicity (all-or-nothing transactions), **C**onsistency (constraints always hold), **I**solation (concurrent txns don't corrupt each other), **D**urability (committed data survives a crash). Postgres gives all four; it's why the source of truth is Postgres, not Redis.

Q What's the N+1 problem and your fix?

A Fetching N parents then lazily loading each one's children = N+1 queries. Fix: eager-load with `selectinload`, which loads all children in one `IN` query. DocuBrain uses it for quiz items and attempts.

Q Why two DB drivers (asyncpg + psycopg2)?

A asyncpg is async — for the FastAPI event loop. psycopg2 is sync — for the Celery worker, which is a plain process with no loop. Right driver for each runtime.

Q What does an index actually do, and the cost?

A A B-tree index turns an O(n) sequential scan into an O(log n) lookup for filtered/sorted queries. Cost: extra storage and slower writes (the index must be updated on every insert/update). You index the columns you filter/join/sort on — in DocuBrain, every foreign key and `user_id`.

Q Explain the commit-before-slow-IO pattern you used.

A An AsyncSession holds a DB connection from its first query until commit/close. During a 30s LLM call that connection sits idle and asyncpg eventually closes it, breaking the later write. So I commit (release the connection to the pool) *before* the slow call; the session leases a fresh connection for the final insert. "Acquire late, release early."

Q PgBouncer transaction mode — what breaks?

A Session-scoped features: SET/RESET, LISTEN/NOTIFY, and session-level prepared statements. I disable asyncpg's prepared-statement cache to be compatible, and run Alembic migrations directly against Postgres (they need a persistent multi-statement session).

Q How do you handle DB schema changes safely?

A Alembic migrations — numbered, reversible, applied with `alembic upgrade head` in the container entrypoint before the app starts, and tested in CI against a real Postgres so a broken migration fails the build.

Q SQL vs NoSQL — why SQL here?

A The data is highly relational with integrity requirements (cascading ownership graphs). Postgres gives FKs, transactions, JSONB for the semi-structured bits (quiz options), and powerful aggregates. I do use a NoSQL-ish store where it fits — Pinecone for vectors, Redis for cache — so it's polyglot persistence, each store for its strength.

Q What's an optimistic vs pessimistic lock?

A Pessimistic = lock the row up front (`SELECT FOR UPDATE`), others wait. Optimistic = don't lock; check a version/timestamp at write time and retry on conflict. DocuBrain's writes are per-user and rarely contended, so neither is needed — but the framing matters for high-contention designs.

6.3 Authentication & Security

Q Why hash passwords with bcrypt and not SHA-256?

A Fast hashes (SHA-256) are *too* fast — an attacker with the hash file can try billions/sec. bcrypt is deliberately slow and salted, with a tunable work factor, so brute-forcing stolen hashes is computationally painful. Use a password-hashing function, not a general-purpose digest.

Q What's a salt and why does it matter?

A A random per-password value mixed into the hash so two users with the same password get different hashes. It defeats precomputed rainbow-table attacks. bcrypt embeds the salt in its output automatically.

Q JWT vs server-side sessions?

A JWTs are stateless — verify the signature, no DB lookup, scales horizontally with no shared session store. The downside is you can't easily revoke one before it expires. DocuBrain gets the best of both: stateless JWT for the frequent access path, plus a revocable Redis-backed refresh token for session control.

Q How does refresh-token rotation detect theft?

A Each refresh deletes the old token and issues a new one. If an attacker steals a refresh token and the legit user refreshes first, the attacker's now-rotated token misses in Redis → 401. The replay is caught.

Q Walk through the OWASP issues you defend against.

A Injection — parameterised ORM queries, never string-built SQL. **XSS** — refresh token in an httpOnly cookie JS can't read; Nginx `nosniff`. **Broken auth** — bcrypt + short JWTs + rotation + generic login errors (no user enumeration). **Security misconfig** — non-root container, secrets in env, `/metrics` blocked publicly. **Unrestricted upload** — magic-byte MIME check, size cap, rate limit.

Q Why magic-byte checking instead of trusting the extension?

A File extensions and the client-sent Content-Type are trivially spoofed. `python-magic` reads the actual file signature, so a malicious executable renamed `.pdf` is rejected because its bytes aren't a PDF.

Q What is CORS and how is it configured?

A Browsers block cross-origin requests unless the server opts in. DocuBrain uses an explicit origin *allowlist* (never with credentials) and restricts methods/headers. It's a browser-enforced policy, not a server firewall — important nuance. *

Q httpOnly, Secure, SameSite — what do they do?

A httpOnly: JS can't read the cookie (anti-XSS). **Secure:** only sent over HTTPS. **SameSite:** controls cross-site sending (anti-CSRF). DocuBrain sets all three on the refresh cookie, tightening them in production.

Q Why is the /metrics endpoint blocked at Nginx?

A It exposes internal telemetry (error rates, endpoint latencies, in-flight counts) that helps an attacker map the system. Prometheus scrapes it from inside the private network; the public edge returns 403.

6.4 RAG, Embeddings & LLMs

Q Explain RAG in two sentences.

A Retrieval-Augmented Generation: instead of relying on the model's training, you retrieve the most relevant chunks of *your* data and put them in the prompt as context. The model answers from that grounded context, so it's accurate, updatable, and can cite sources.

Q What is an embedding?

A A vector (768 numbers here) representing a piece of text's meaning, positioned so semantically similar text is geometrically close. It turns "meaning" into "distance," which makes semantic search a nearest-neighbour problem.

Q Why chunk documents, and how big?

A Embedding models and prompts have size limits, and retrieval is more precise on small focused passages. ~1500 chars balances precision against losing context, with 100-char overlap so meaning isn't cut at chunk boundaries.

Q Cosine similarity vs Euclidean distance?

A Cosine compares direction (angle), ignoring magnitude — ideal for text where direction encodes meaning and magnitude is mostly length noise. It's the standard metric for embedding search.

Q How do you stop one user retrieving another's documents?

A Every vector is stored with a `user_id` in its metadata, and every Pinecone query filters by the requester's `user_id`. Tenant isolation enforced at the vector-search layer.

Q How do you reduce hallucination?

A The prompt explicitly instructs "answer only from the provided context," retrieval grounds it in real passages, and sources are reported. If nothing relevant is retrieved, the context says so and the model is steered to admit it doesn't know rather than invent.

Q RAG vs fine-tuning?

A Fine-tuning changes weights — costly, slow to update, no citations, impractical per-user. RAG keeps knowledge external and instantly updatable, grounds answers, and cites. For per-user documents, RAG is the only viable approach.

Q How do you handle non-deterministic LLM output for quizzes?

A Force JSON mode (`response_mime_type="application/json"`), then defensively validate every item — drop malformed questions, clamp out-of-range indices, return a clean 502 if the whole response isn't valid JSON. Never trust the structure blindly.

6.5 Distributed Systems & Messaging

Q Why a message queue at all?

A To decouple a slow producer from a fast response. Upload returns instantly by pushing a job to Redis; the Celery worker processes it independently. It also smooths bursts (the queue buffers) and lets you scale workers separately from the API.

Q At-least-once vs exactly-once delivery?

A Exactly-once is essentially unachievable across a network. Celery with `acks_late` gives at-least-once — a task may run twice if a worker dies mid-job. The fix is idempotency.

Q How are your tasks idempotent?

A Re-processing a document re-embeds and upserts to the *same* deterministic Pinecone IDs (`{doc_id}_{i}`), overwriting in place. Running twice yields the same end state — no duplicate vectors.

Q How does the worker talk to the API for real-time updates?

A They're separate processes, so they can't call each other. The worker PUBLISHes to a Redis channel; the API process holding the user's WebSocket is SUBSCRIBed and relays it. Pub/Sub decouples them and scales horizontally.

Q Explain the CAP theorem in your context.

A Under a network partition you choose Consistency or Availability. Postgres (primary) is CP — it favors consistency. The read replica introduces availability/scale at the cost of eventual consistency (lag), so I only send staleness-tolerant reads there and keep read-your-writes on the primary.

Q What is a circuit breaker and why three of them?

A A state machine that fails fast when a dependency is unhealthy, preventing cascading failures and giving it room to recover. Three independent breakers (Gemini/Pinecone/S3) so one slow dependency doesn't trip the others — and each is tuned to its recovery profile.

Q How would you make this multi-instance?

A The API is already stateless (JWT auth, state in Postgres/Redis), so you just run more containers behind Nginx — add lines to the upstream block. Redis Pub/Sub already handles cross-instance WebSocket fan-out. The DB tier scales via PgBouncer + read replicas. Nothing in the app assumes a single instance.

6.6 Caching

Q What caching strategy do you use?

A Cache-aside (lazy loading): on read, check Redis; on miss, hit Postgres and write the result back with a TTL. Document lists 30s, chat-session lists 60s. The app, not the cache, owns the read-through logic.

Q How do you handle cache invalidation?

A Write-through invalidation: any mutation (upload, delete, new chat session) deletes the relevant cache key so the next read repopulates fresh. Short TTLs are a safety net so even a missed invalidation self-heals quickly.

Q "There are only two hard problems..." — how do you avoid stale data?

A Two layers: explicit invalidation on every write path, plus short TTLs as a backstop. For data that must be fresh (read-your-writes), I don't cache at all — correctness over a few ms of latency.

Q What's a cache stampede and do you risk it?

A When a hot key expires and many requests simultaneously miss and hammer the DB. At DocuBrain's scale (per-user keys, low concurrency per key) it's negligible; at scale you'd add jittered TTLs or a lock/"single-flight" so only one request rebuilds the key.

6.7 Networking, HTTP & Real-Time

Q What does a reverse proxy do that the app can't?

A TLS termination, edge rate limiting, load balancing, response buffering (shielding workers from slow clients), gzip, and being the single hardened internet-facing surface. It does the cross-cutting network concerns so the app does business logic.

Q SSE vs WebSocket vs long-polling?

A SSE = one-way server→client streaming over plain HTTP, auto-reconnect — perfect for chat token streaming. WebSocket = full-duplex, for true bidirectional/push — used for notifications. Long-polling = the legacy fallback (repeated requests held open); both SSE and WS are strictly better. I picked each by directionality.

Q Why does the WebSocket pass the token as a query param?

A Browsers can't set custom headers (like `Authorization`) on a WebSocket handshake. So the JWT goes in `?token=`, and it's validated *before* the socket is accepted — an invalid token never opens a connection.

Q How does Nginx hold thousands of connections cheaply?

A It's event-driven: each worker uses Linux `epoll` to watch many sockets in one process, reacting only when one is ready (O(1)), instead of one thread per connection. That's the architecture behind handling 10k+ idle/slow clients.

Q What status codes do you return and why?

A 422 for validation (Pydantic, automatic), 401 vs 403 (unauthenticated vs unauthorized), 404 for not-found *and* not-owned (don't leak existence), 429 for rate limits (with `Retry-After`), 503 when a circuit breaker is open, 502 for a bad upstream/AI response. Semantically correct codes let clients react correctly.

6.8 System Design & Trade-offs

Q What's the single biggest bottleneck and how would you scale past it?

A The AI calls (Gemini) — they're the slow, rate-limited, costly path. I already move document embedding off-request into Celery and stream chat so it feels fast. To scale: more Celery workers for ingest throughput, a semantic cache for repeated questions, batching embeddings, and per-user quotas. The web/data tier scales trivially by comparison (more containers, PgBouncer, replicas).

Q How would you handle 100× the traffic?

A Horizontally scale stateless API containers behind Nginx; scale Celery workers independently; add Postgres read replicas and route reads to them; the cache absorbs read load; Pinecone and S3 are managed and scale themselves. The architecture was built to scale out — nothing holds local state.

Q What would you do next if you had a week?

A Finish Phase 3: wire correlation IDs into OpenTelemetry for distributed tracing and add Sentry for error aggregation. Then a semantic response cache for chat, and automated rollback in the deploy pipeline (the hook is already stubbed).

Q What's a design decision you'd revisit?

A The fixed-window rate limiter allows a boundary burst; at higher scale I'd move to a sliding-window or token-bucket in Redis. And I'd consider pgvector to collapse Pinecone into the Postgres I already run, trading zero-ops for cost and data locality — it's a localized swap because vector access is behind one infra client.

Q How do you test this system?

A CI runs lint → pytest against real Postgres/Redis service containers (after applying migrations) → build → deploy → a post-deploy k6 smoke test against live prod. Load is validated with k6 (50 VUs, p95 31ms, 0% errors). Real dependencies over mocks for integration confidence.

Q If the database goes down, what happens?

A Writes and most reads fail fast (the app surfaces a 5xx rather than hanging, thanks to connection timeouts). Cached reads may still serve from Redis until TTL. On recovery, `pool_pre_ping` transparently replaces dead connections — no restart needed. The startup probe also retries with exponential backoff so a slow-booting DB doesn't crash the app.

One-Page Rapid-Fire Cheat Sheet

The last thing to skim before you walk in.

If they say...	You say...
"Walk me through the architecture"	Browser → Nginx (edge: TLS, rate limit, LB) → FastAPI (async) → services → repos → Postgres via PgBouncer. Slow work → Celery via Redis. Vectors → Pinecone. Files → S3. Real-time → Redis Pub/Sub + WebSocket.
"Why async?"	I/O-bound workload; one thread serves thousands of waiting requests; blocking SDKs are pushed to threads.
"Why two DB drivers?"	asyncpg for the async API, psycopg2 for the sync Celery worker.
"Why PgBouncer?"	200 client conns → ~10 real Postgres conns, transaction mode. Postgres connections are expensive OS processes.
"Redis does what?"	Four jobs: cache, Celery broker, rate-limit counters, Pub/Sub for WebSockets.
"Auth model?"	15-min access JWT (stateless) + 7-day random refresh token in Redis via httpOnly cookie, with rotation for theft detection.
"Explain RAG"	Chunk → embed (768-dim) → Pinecone. Query: embed → top-5 cosine search filtered by user_id → stuff into prompt → stream Gemini → cite sources.
"Resilience?"	Three circuit breakers (Gemini/Pinecone/S3), two-tier rate limiting, retries with backoff, graceful 503s.
"The clever DB trick?"	Commit before slow I/O to release the connection; the session leases a fresh one for the final write.
"Observability?"	Prometheus metrics + structured JSON logs + correlation IDs (ContextVar). Next: OpenTelemetry + Sentry.
"Scale story?"	Stateless API → more containers behind Nginx; workers scale independently; read replicas; managed Pinecone/S3.
"Performance numbers?"	k6: 50 VUs, p95 31ms, 0% errors, 26.5 req/s on the full Dockerised stack.
"Delivery?"	GitHub Actions: lint → test (real PG/Redis) → build (SHA-tagged) → deploy → prod smoke test. Terraform for AWS infra.

YOUR CLOSING LINE

"The theme across the whole project is the same: **fast path stays in the request, slow path moves out of band, and every external dependency is wrapped so one failure can't cascade.** That's what makes an AI prototype behave like a production backend."